Proyecto de trabajo anual del grupo GAMY: “Multiplataformero”. Informe de avances

Di Guglielmo Luca, Veit Alexander, Hlynczuk Benjamin & Fricker Julian

Escuela de Educación Técnica N°1

Laboratorio de Programación 5°3°

Ganduglia Yamil & Muñoz Elias Mansilla York

30/04/26

Tabla 1

Registros de versiones del informe anual de G.A.M.Y

Versión	Fecha	Descripción del cambio	Autor	Estado
v.0.1.0	30 / 04 / 2026	Adición título e índice	Luca Di Guglielmo	Borrador
v.0.1.1	15 / 05 / 2026	Adición de introducción	Luca Di Guglielmo	Borrador
v.0.1.11	02 / 09 / 2026	Modificación de estructura y informacion	Luca Di Guglielmo	Borrador
v.0.2.0	15 / 09 / 2026	Adición de registro de versiones (este gráfico)	Luca Di Guglielmo	Borrador
v.0.2.1	3/ 11 / 2026	Nuevo nombre para el Informe	Alexander Veit	Borrador
v.0.2.2	06 / 11 / 2026	Adición de información en Desarrollo	Luca Di Guglielmo	Borrador
v.0.2.3	11 / 11 / 2026	Adición de Información hasta la conclusión del PIA	Luca Di Guglielmo	Release

Tabla de Contenido

Introducción.....3

1.1 - ¿Cuál va a ser nuestro proyecto?.....3

Método.....5

2.1 - ¿Qué herramientas vamos a utilizar?.....5

2.2 - Investigación y aprendizaje de software.....6

2.3 - Proyecto N°1 ‘Arepa Ride’.....6

2.4 - Cambio de proyecto.....8

2.5 - Multiplataformero.....9

Introducción

1.1 - ¿Cuál va a ser nuestro proyecto?

Nuestra idea empezó con el juego como trabajo anual, este debería ser fundamentalmente simple, con código basado en conceptos básicos y sin la introducción de mecánicas complejas sin razón.

Para lograr esto, primero pensamos en que sería nuestro proyecto, la primera idea fue un shooter en 3D con habitaciones generadas procedimentalmente (generadas a partir de lógica y algoritmos), el problema con esta fue que significaba mucha experiencia y conocimiento complejo al cual nosotros no estábamos familiarizados.

Entonces optamos por una opción más ligera, un RPG, Top-Down en 2D el cual adoptaría un combate que integra elementos de RPGs como Earthbound & Final Fantasy IV (Uso de items, armaduras y diferentes herramientas) y bullet-hells, creando un combate similar al de juegos como Deltarune y Undertale. Luego de definir los pilares del ciclo de juego, discutimos en el aspecto del juego, para esto decidimos en un arte basado en sprites de 16x16 bits, dándole un estilo “retro” al juego, esto no le quita la posible calidad artística que este puede tener.

Al llegar el segundo cuatrimestre, nos dimos cuenta que nuestro deseo de un RPG top-down con todas las características que se describieron es muy difícil y extenso para el nivel en el que nosotros estamos, lo que nos llevó a una decisión difícil, cambiar de proyecto. Por suerte, uno de nuestros desarrolladores, Benjamin Hlynczuk, estaba creando un juego de plataformas 2D chico, entonces, decidimos usar este como base y expandirlo para volverlo nuestro proyecto. En conclusión, nuestro proyecto es un juego de plataformas (i.e Cat Mario, Super Mario Bros) que busca en su base ser difícil e impredecible, el estilo y las texturas (sprites), como con el RPG, serán de 16x16 píxeles, adoptando un estilo el cual es simple de desarrollar pero admite tener un estilo de calidad.

Método

2.1 - ¿Qué herramientas vamos a utilizar?

Una vez establecida la idea general del juego, pasamos a buscar software que nos permita volver esta idea en un proyecto real.

Lo primero que necesitábamos era un motor de juego, para esta tarea tuvimos tres candidatos: Unreal Engine, capaz de correr hasta los gráficos más impactantes y realistas, pero extremadamente pesado, necesidad de hardware de alto rendimiento y está ambientado para la creación de juegos en 3D, no 2D. Unity, buen motor y usado por muchas personas / grupos, pero generaría problemas al querer publicar el juego por nuevos requerimientos establecidos recientemente.

Esto nos deja con la última opción; Godot Engine, código abierto, ligero, mantiene y puede usarse para proyectos en 3D y 2D y código fácil de aprender, ya que está directamente inspirado en Python. Luego, para el arte decidimos utilizar un sitio web llamado “Piskel” y la edición de archivos .html, .css y .js en Visual Studio Code como IDE oficial del proyecto (excluyendo el desarrollo del juego, el cual se hará en VSC)

2.2 - Investigación y aprendizaje de software

Al establecer el software y lenguajes que utilizamos en este proyecto surgió un problema que abarcara el tiempo de los primeros meses de desarrollo, el aprendizaje de Godot Engine, GDscript, Git, Github y el desarrollo de videojuegos y web. Para aprender GDscript y Godot Engine, se optó por empezar con tutoriales textuales y multimedia en plataformas como Youtube y Godot Docs. Estos medios nos dieron el conocimiento básico para empezar con nuestro proyecto. Este aprendizaje fue emprendido en su totalidad por el grupo como entidad individual, ya que a lo largo de los primeros los profesores encargados dieron total prioridad al Proyecto Integrador Anual (P.I.A), dejando todo el desarrollo de nuestro aprendizaje de programación a manos del estudiante, sin realmente enseñarnos, ni siquiera algunos considerados como repaso

2.3 - Proyecto N°1 'Arepa Ride'

Gracias a lo adquirido durante los primeros meses y la presión que nos ponía el primer avance anual, decidimos crear el proyecto .godot a mediados de abril. Lo primero que creamos fue un sistema básico de movimiento, que registre la entrada del jugador y a partir de eso mover a un modelo de prueba a lo largo de un plano [Figura 1].

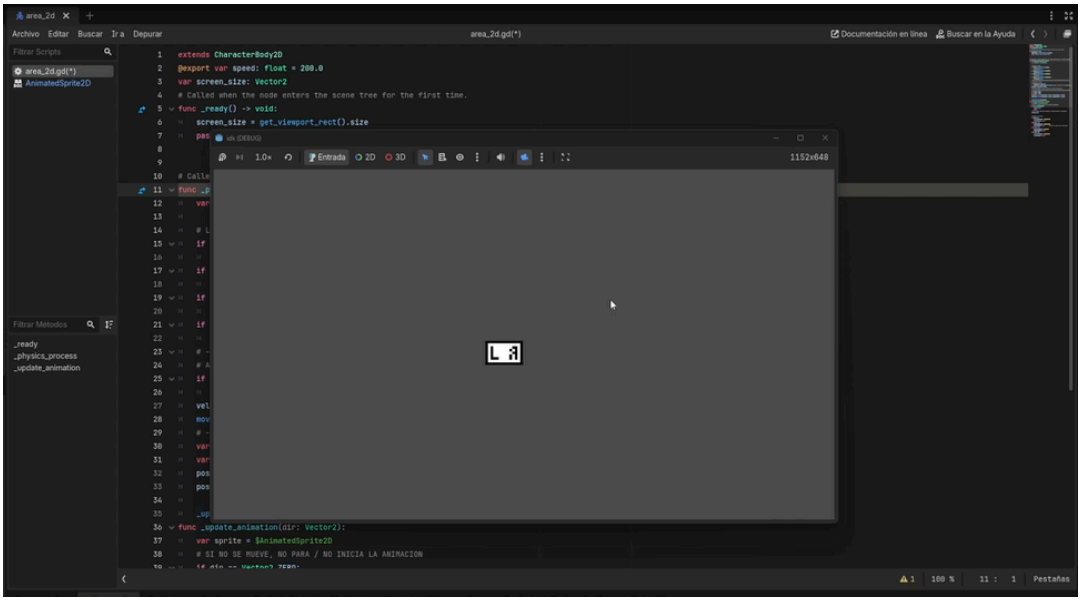


Figura 1 - Una de las primeras versiones del P.I.A,

El paso siguiente tras la implementación del sistema de movimiento fue darle al jugador un espacio por el cual moverse y que la cámara lo siga, además de lo antes mencionado, añadimos un nuevo modelo de jugador [Figura 2] [Figura 3]. Semanas antes del primer avance, logramos añadir un inventario, objetos para este mismo y ‘cofres’ que de donde en un futuro podrias conseguir los objetos.

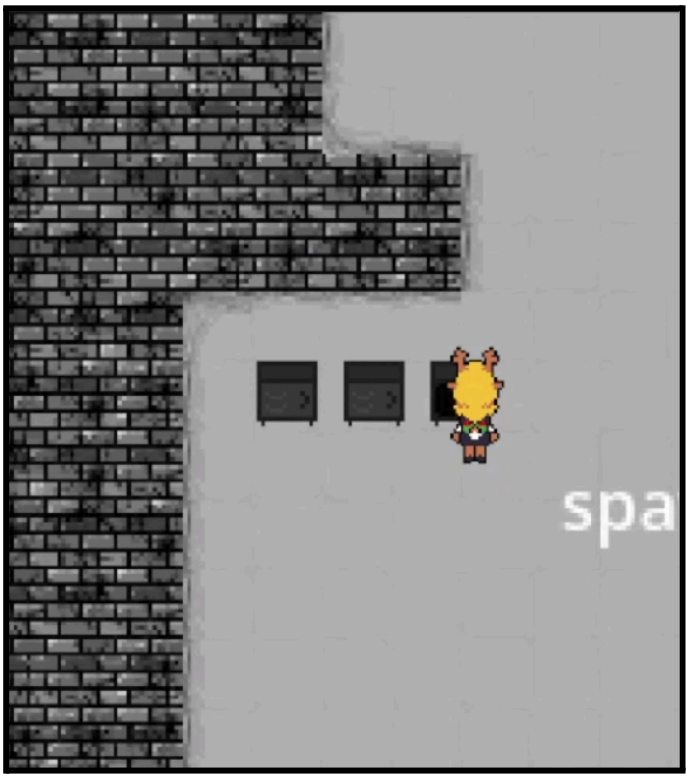


Figura 2 - Cofres



Figura 3 - Sistema de Inventario

A lo largo de todo este proyecto como grupo, a pesar de la habilitación y el uso extensivo de los mismo profesores de esta herramienta, evitamos abusar el uso de inteligencia artificial, buscando en este proyecto hacer un trabajo simple y ‘sencillo’ para aumentar nuestro conocimiento sobre el desarrollo de videojuegos, un tema que nos interesa. Pero durante el desarrollo de Arepa - Ride, nos dimos cuenta de que nuestra propuesta parecía cada vez más difícil y extensa.

2.4 - Cambio de proyecto

Como fue mencionado en la sección anterior, el desarrollo de Arepa - Ride se entorpecia por cada semana que pasaba, la implementación de nuevas mecánicas era cada vez más difícil y tardado. Debido a esto, a finales del primer cuatrimestre e inicios del segundo nos planteamos el cambio del proyecto anual integrador. Primero debíamos pensar ¿Por qué cambiamos de proyecto? Esto sucedió en base a 3 factores principales, las primeras que abordaremos, la complejidad y la falta de experiencia.

Aunque al principio la implementación de mecánicas resultó fácil, el avance del desarrollo trajo junto a él nuevos problemas que resultaron de un nivel muy avanzado para lo que nosotros estábamos acostumbrados. A lo largo del proyecto nosotros nos apoyamos en base a tutoriales en formato de video y texto, para el aprendizaje del lenguaje GDscript y su motor, Godot Engine, en base a estos mismos creamos la mayor parte de las mecánicas, código el cual funcionaba y entendíamos hasta cierto punto. Esto mismo genera el mayor problema de todos, cuando era necesario relacionar mecánicas, por ejemplo, el inventario y el sistema de cofres, se nos volvió una misión imposible, esto sucedió gracias a que, aunque entendiamos la lógica detrás del código, al fin y al cabo este no completamente nuestro, volviendo al código casi imposible de escalar, nuestro proyecto era muy ambicioso para nuestra habilidad. Estas ‘excusas’ solas no tienen mucho sentido por sí solas como razón para cambiar el proyecto, por lo tanto pasaremos a hablar sobre el último y muy importante factor, el tiempo límite.

Si repasamos los factores mencionados previamente, estos no son problemas sino cosas que ralentizan el desarrollo, esto significa que con un tiempo ilimitado o simplemente largo que sea usado para tan solo la creación de Arepa - Ride. Pero este no fue el caso, 4 horas por semana era el tiempo total que teníamos asignado a la materia, hay que tener en cuenta también que programación no es la única materia de la carrera, muchas materias pedían trabajos, evaluaciones y exposiciones al mismo tiempo, lo que dificulta mucho el trabajo y ponía al PIA ‘de fondo’ en relación de urgencia.

Nuestro tiempo también se vio reducido hasta llegar a momentos en que no teníamos ni la oportunidad de trabajar sobre el proyecto cuando aparecieron las actividades / trabajos prácticos. Este grupo de 9 actividades nos dejaba con muy poco tiempo para trabajar sobre el proyecto y aprender Godot Engine, también nos ponía aún más presión gracias a que los tiempos que teníamos para trabajar eran usados sobre estas en vez del proyecto.

2.5 - Multiplataformero

Por las razones descritas anteriormente, decidimos cambiar nuestro proyecto 'Arepa - Ride' por uno nuevo, de más fácil desarrollo, 'Multiplataformero', este es un 'rage - game' de plataformas en 2D (similar a títulos de franquicias como Super Mario) que consiste de 5 niveles, para progresar, el jugador debe recolectar todas las monedas de un escenario / nivel.

¿Qué es un 'rage - game'? Un rage - game busca como su principal objetivo hacer enojar al jugador, de eso viene su nombre, esto lo logra utilizando mecánicas 'injustas' para el jugador forzandolo a perder y volver a intentar para poder progresar, por ejemplo, en 'Multiplataformero' hay bloques específicos en cada nivel que caen y 'matan' al jugador cuando caen sobre él.

El desarrollo del multiplataformero, a comparación de Arepa Ride, fue mucho más rápido, teniendo una versión jugable y casi definida a las pocas semanas de desarrollo, manteniendo los mismos estándares y evitando el uso de IA.

Conclusión

A lo largo de este año, realizamos un proyecto el cual fue iniciado, desarrollado y terminado completamente gracias al esfuerzo de nuestro grupo, G.A.M.Y. Logramos y debimos como grupo aprender plenamente por nuestra cuenta, sin ningún tipo de apoyo previo más que documentos y ‘actividades’ que embarraron el desarrollo.

Referencias

Repositorio Github: https://github.com/specIux/GAMY-project/tree/main

Repositorio Gitlab: https://gitlab.com/gamy-group2/GAMY-project